SEMINAR SUMMARY:
FEBRUARY 17, 2026 - MARCO ALDI
NP PROBLEMS AND THEIR QUANTUM ANALOGUES

ROB TOMPKINS

Anything that can be calculated on today's computers could have been calculated using mechanical machine. An example of a particularly sophisticated calculating machine was established by Babbage. In fact anything calculable with one of the most sophisticated computers today (quantum aside) could be worked out by hand in a step-by-step process with boundaries given for input and a sufficient set of instructions. These mechanical calculators or computers as we call them, are particularly good at solving a particular set of problems. These are the easy problems with which we are familiar, alphabetizing a list, finding the value of an element in a list, merging two lists, and many more. Though things become difficult when we are asked if an arbitrary collection of integers contains a subset that sums to zero. Moreover, it is even harder to find that subset. Note, there are a collection of problems that are essentially analogous this problem about finding a subset of integers that sums to zero.

In the early 1970's, 1971 to be exact, Stephen Cook and Leonid Levin independently went about attempting to formally define the characteristic to a problem that lands it in one of these two categories. The easier first collection of problems are those that given an input of length n can be solved in $n^k + a$ steps with $a, k \in \mathbb{N}$. These problems we call *polynomial time* problems because they can be solved in $\mathcal{O}(n^k)$ steps the problem space $\mathbf{P}$.

Then there are problems that are known to take a substantial amount of time, for example exhaustive search for a directory in a large un-indexed directory tree. These problems are known to take $\mathcal{O}(2^n)$ steps, or *exponential time.*

Now, these inbetween difficulties problems. How are we to classify them. Originally, we were concerned with polynomial time algorithms over a Non-deterministic Turing Machine. Where a Non-deterministic Turing Machine is one that given a tape position and a state, one has multiple next states into which to move, which leaves us with a relatively vague yet faster path to our solution. Eventually people became impatient with the vagueness of this definition, and the following definition arose.

Definition 1. (The class $\mathbf{NP}$) A language $L \subseteq \{0,1\}^*$ (the kleene closure over $\{0,1\}$) is in $\mathbf{NP}$ provided there exists a polynomial $p : \mathbb{N} \to \mathbb{N}$ and a polynomial-time turing machine M

Date: February 19, 2026.

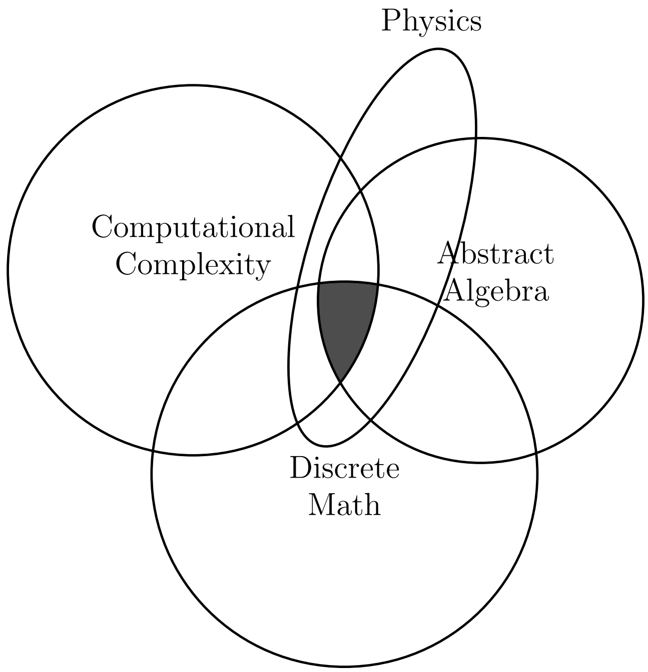

FIGURE 1. Fields involved in making progress on **NP**-hard problem research

(called the *verifier* of L) such that for every $x \in \{0,1\}^*$

$$x \in L \Leftrightarrow \exists u \in \{0,1\}^{p(|x|)} \quad \text{s.t.} \quad M(x,u) = 1.$$

If $x \in L$ and $u \in \{0,1\}^{p(|x|)}$ satisfy $M(x,u) = 1$, then we call u a *certificate* for x (with respect to the language L and the machine M).

Question 1. *Does* $\mathbf{P} = \mathbf{NP}$*?*

Over the years multiple different fields have all made their own progress on the problem. Figure 1 gives us a venn-diagram of some of the more important fields involved in making progress on **NP**-hard problems, whether **P** and **NP** are the same set of problems.

Now that we have the class of **NP**-hard problems, it becomes important to have an example of an **NP**-hard problem. Now we earlier mentioned the integer subset sum problem, but this was not the original problem proven to be NP-hard. That was the boolean satisfaction problem, "SAT". SAT is defined by considering a list of variables $s_0, s_1, \ldots, s_{n-1} \in \{0,1\}$ and a function $f : \{0,1\}^n \to \{0,1\}$, find the answer to $f(s_0, s_1, \ldots, s_{n-1}) = 0$ (or 1). We can think of f as an arbitrary list of and's $\wedge$ and or's $\vee$, with our s's as variables, and we are trying to find a solution that yields *true*. Clearly finding a solution is quite difficult, but checking if a collection of s's is a solution is an answer is as easy as plugging them into f, a polynomial-time task. Oddly, every *known* **NP**-hard problem reduces to SAT, and we call these problems **NP**-complete. Four classical examples of problems that are NP-complete are: the Hamiltonian Path problem, 3SAT (the boolean satisfaction problem factored into 3

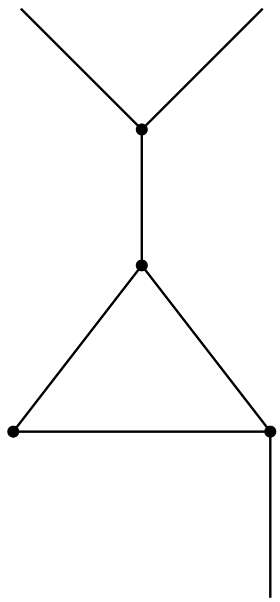

FIGURE 2. A graph

blocs of ands or'd together or vice versa), the Independent Set Problem, and the Travelling Salesman Problem.

We went over the definition of the Independent Set problem (which personally I had not seen before and found quite interesting). We consider graphs of the form given in Figure 2, and the following definition.

Definition 2. A subset of vertices $S \subseteq V$ in a graph $G = (V, E)$ is an *independent set* for all $u, v \in S$ if the edge $\{u, v\} \notin E$.

The decision problem is stated as: given a graph G and $k \in \mathbb{Z}$, does G contain an independent set of size $\geq k$?

1. The RSA Algorithm

(1977, Rivest–Shamir–Adleman) We briefly went over the RSA algorithm which is one of the most widely used algorithms on the planet and furthermore it's used for transmitting sensitive data. Essentially (without the modular arithmetic work), it's a public private key process where the private key is two large prime numbers p, q and the public key is $pq = n$ their product. On the other hand, it is still not known whether integer factorization is **NP**-hard, so we have the whole of cryptography (yes mainly talking about https). Thought, there has been found to be a polynomial time algorithm for integer factorization in the quantum computing realm. That said, by 1994 because of the work of Peter Shor, we got an algorithm for more quickly factorizing integers

2. Quantum Computing

So what is a Quantum Computer? Well, effectively it works in the same manner as a turing machine except instead of having a finite set of states for the machine and only $\{0,1\}$ as the alphabet for our tape, we have that both the set of states and the tape alphabet are Hilbert spaces with the tape alphabet being a Hilbert space where the value on the tape is given by

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

a linear combination of these two bipolar states represented on the blok sphere which we refer to as a *qubit*.

Regarding quantum computing, we know a few things: first thing is that we have a quantum analogue to the boolean satisfaction problem, which is referred to as the Quantum Merlin Arthur complexity class or QMA for short. We naturally start building a list comparing quantum computing and it's natural analogues

Classical	Quantum
$\{0,1\}$	$\alpha\|0\rangle + \beta\|1\rangle$
NP	**QMA**
$\vdots$	$\vdots$

One might naturally want to extend this table to include the Karp list. A reasonable attempt at creating a mechanic or process by which to extend this list began in 2004 with the work of Dani, Meera, and Mainkar. The idea is to find a linear transformation $\mathcal{L}$ from the graph space to the 2-step nilpotent Lie algebra space such that isomorphism is preserved. Note that the graph space is generally fully embedded within the Lie algebra space.

Lastly and most importantly in 2024, a linear transformation $\mathcal{L}$ was found and defined for the independent set problem. And, Dr. Aldi along with Gharibian and Rudolph proved that there exists a quantum analog for the independent set problem, the first on the Karp list to make it over into the quantum space.